

AGENTIC AI COURSE

Class 01: AI Basics

We Don't Teach, We Educate

Instructor	Telusko
Website	telusko.com
Topic	AI fundamentals, NLP, Transformers, and LLMs
Level	Beginner to Intermediate

These notes summarise Class 01 of the Telusko Agentic AI Course. They cover AI fundamentals, the NLP pipeline, word embeddings, neural networks, Transformers, and large language models. Read them top to bottom. Each part sets up the next one.

1. What is Artificial Intelligence (AI)?

Artificial Intelligence is the field of building systems that can do tasks which normally need human intelligence. Think learning, reasoning, problem solving, and decision making. The goal is not to copy a human. The goal is to build machines that can handle useful tasks on their own.

KEY POINT

AI is the ability of machines to perform tasks that normally require human intelligence.

AI vs Traditional Software

Traditional Software	AI Systems
Input plus rules give the output	Data plus learning give predictions
Logic is fixed and written by hand	Patterns are learned from data
Breaks when the input changes	Adapts and generalises to new data
Stays the same over time	Improves as it sees more data

The AI Hierarchy

Here is how the pieces fit together.

AI → ML → DL → NLP → Transformers → LLMs → AI Agents

- **AI:** making machines intelligent.
- **Machine Learning (ML):** learning patterns from data.
- **Deep Learning (DL):** ML built with neural networks.
- **Natural Language Processing (NLP):** working with human language.
- **Transformers:** the architecture behind modern NLP.
- **Large Language Models (LLMs):** models like GPT, Claude, and Gemini.
- **AI Agents:** LLMs that act on their own using tools, memory, and planning.

READ THIS CAREFULLY

These layers are not all strict subsets of one another. ML sits inside AI, and DL sits inside ML. But NLP is a field that is older than deep learning. Today it is powered by deep learning and Transformers, which is why it sits here.

Generative AI (GenAI) is the broad family of models that create new content. LLMs are one part of that family. Agentic AI sits at the very top of the stack.

KEY INSIGHT

Agentic AI is LLMs plus tools plus memory plus planning. You need a clear picture of every layer before you start building agents.

2. Natural Language Processing (NLP)

NLP lets computers read, process, and produce human language. It is the bridge between how people talk and how machines compute.

Common NLP Applications

Application	Example
Conversational AI	ChatGPT, Claude, Gemini
Search engines	Semantic ranking in Google Search
Machine translation	Google Translate, DeepL
Voice assistants	Siri, Alexa, Google Assistant
Chatbots	Customer support automation
Sentiment analysis	Product reviews, social media

Core NLP Tasks

Tokenization: breaking text into smaller units called tokens. Words, tokens, and characters are not the same thing. LLMs work on tokens.

I love Java → [I] [love] [Java]

Modern LLMs use subword tokenization, for example Byte Pair Encoding. Rare or long words get split into pieces. So the word unhappiness can become [un] [happiness], which is 2 tokens.

POS Tagging: giving each word its grammatical role.

Rahul (Noun) plays (Verb) cricket (Noun)

NER (Named Entity Recognition): finding real world named entities in text.

Rahul → Person | Infosys → Org | Bangalore → Location

Sentiment Analysis: detecting the emotion or opinion in text.

This course is amazing → Positive

Text Classification: putting text into set categories.

Email → Spam or Not Spam | Article → Sports, Tech, or Politics

IMPORTANT

Tokens are not words. LLM pricing and context windows are measured in tokens, not words. When you think about cost and limits, always think in tokens.

3. Word Embeddings: Teaching Computers Language

Computers only understand numbers. Embeddings turn words into dense number vectors that carry meaning. Words with similar meaning end up with similar vectors.

Word → Vector → Math on meaning

A famous example: King minus Man plus Woman is close to Queen. You can do arithmetic on meaning.

Word2Vec	GloVe
Uses local context, the nearby words	Uses global context, the whole corpus
Learns from a sliding window	Learns from a co-occurrence matrix
Produces static embeddings	Produces static embeddings
Fast to train	Strong on word relationships
One vector per word, always the same	Often a bit better on analogy tasks

NOTE

Both Word2Vec and GloVe give static embeddings. The same word gets the same vector no matter the sentence. So bank in river bank and bank in bank account share one vector. Transformers fixed this later with context aware embeddings.

4. Neural Networks and Deep Learning

Neural networks take their idea from the brain. They are layers of artificial neurons joined by weighted connections. Deep Learning just means neural networks with many layers.

Anatomy of a Neuron

A single artificial neuron works like this:

- **Inputs ($x_1, x_2, \dots, x_n$):** raw features, or outputs from the previous layer.
- **Weights ($w_1, w_2, \dots, w_n$):** how important each input is. These are learned.
- **Bias (b):** shifts the activation threshold.
- **Weighted sum:** $z = \sum(w_i \cdot x_i) + b$
- **Activation:** output = $f(z)$. This adds nonlinearity.
- **Output:** passes to the next layer.

Activation Functions

Function	Formula	Use case
ReLU	$\max(0, x)$	Hidden layers, most common
Sigmoid	$1 / (1 + e^{-x})$	Binary classification output
Softmax	$e^{(x_i)} / \sum e^{(x_j)}$	Multi-class output
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	RNNs, centred around 0

Why Deep Learning Beat Classical NLP

Classical NLP	Deep Learning NLP
Hand-written rules	Learns straight from data
Keyword matching	Understands meaning
Manual feature engineering	Automatic feature extraction
Hard to scale	Scales with data and compute
No real context	Rich contextual understanding

5. Transformers: The Architecture That Changed Everything

Transformers came from the 2017 paper Attention Is All You Need by Vaswani and colleagues. They dropped recurrence and let tokens attend to one another directly, instead of reading strictly one step at a time. That gave two big wins. Training runs in parallel, and the model can link words that sit far apart.

The Transformer Pipeline

Text → Transformer → Prediction

1. **Text input:** raw natural language.
2. **Tokenization:** split into tokens.
3. **Token IDs:** map tokens to integers.
4. **Embeddings:** turn IDs into dense vectors.
5. **Positional encoding:** add word order information.
6. **Self attention:** each token looks at the others.
7. **Multi-head attention:** several attention views in parallel.
8. **Feed-forward layer:** transform the representations.
9. **Output:** a probability for the next token.

Key Concepts

Self Attention: each token builds a weighted mix of the other tokens. The model learns which tokens matter to each other.

The animal did not cross the street because it was tired. → it links back to animal, not street.

WORTH KNOWING

In an encoder, a token can look at every other token, both left and right. In a decoder, a token can only look at the tokens before it. That masking is exactly what lets a decoder generate text one token at a time.

Positional Encoding: Transformers read all tokens at once, so they have no built-in sense of order. Positional encodings add that order back using sine and cosine functions.

Without position info: dog bites man == man bites dog

Multi-Head Attention: several attention heads run in parallel. Each head can focus on a different kind of relationship. They are not voting on one answer.

As a rough idea: Head 1 → subject and verb Head 2 → object Head 3 → location

Treat that example as an illustration only. In real models, some heads do learn clear patterns, but many do not map to a single neat role.

Encoder vs Decoder

Encoder	Decoder
Reads and understands the input	Generates the output text
Looks both ways (bidirectional)	Looks left to right only (causal)
Example: BERT	Example: GPT series
Used for classification and NER	Used for text generation and chat

Encoder plus decoder together: T5 and BART. These are used for translation and summarisation.

KEY INSIGHT

Claude, ChatGPT, and Gemini are decoder-only Transformers. They predict the next token, one at a time, until the answer is complete. The skill comes from scale: billions of parameters trained on huge amounts of text.

6. Large Language Models (LLMs)

LLMs are Transformer models trained on massive text data, with billions or even trillions of parameters. They can write coherent, on-topic text across almost any subject.

How LLMs Work: Key Properties

Next-token prediction: the model is trained to predict the next token from all the tokens before it. Reasoning, coding, and creativity all emerge from this one task at scale.

Context window: the most tokens a model can hold at once. This ranges from about 4K in older models to 1M or more, for example Gemini 1.5 Pro.

Temperature: controls randomness. It usually sits on a 0 to 2 scale, and the exact range depends on the provider. OpenAI and Gemini allow up to 2, while Anthropic caps it at 1. The default is 1. At 0 the model is deterministic and picks the most likely token. Higher values add variety but can hurt accuracy.

Prompt engineering: writing inputs that get the output you want. A core skill for agent work.

Fine-tuning: adapting a pre-trained model to a specific task or domain with extra data.

RLHF: Reinforcement Learning from Human Feedback. It aligns model output with human preferences.

Popular LLMs (2024 to 2025)

Model	Creator	Type	Notable for
GPT-4o	OpenAI	Decoder	Multimodal, widely used
Claude 3.5 Sonnet	Anthropic	Decoder	Safety, long context, coding
Gemini 1.5 Pro	Google	Decoder	1M token context window
Llama 3	Meta	Decoder	Open source, runs locally
Mistral 7B	Mistral AI	Decoder	Efficient, open weights

Note: this table is a snapshot from 2024 to 2025. Newer versions have released since, but the ideas here still hold.

From LLMs to Agents

On its own, an LLM takes text in and gives text out. It cannot click a button, call an API, or remember what happened yesterday. An agent adds the missing parts.

- **Tools:** let the model call APIs, run code, search, or query a database.
- **Memory:** let it carry context across steps and sessions.
- **Planning:** let it break a goal into steps and decide the order.
- **A loop:** perceive, reason, act, observe, then repeat until the goal is done.

WHERE THIS COURSE GOES

That loop is the heart of Agentic AI. The rest of this course builds it step by step, with Java, Spring AI, LangChain4j, and the tools around them.

7. Quick Reference Summary

Term	One-line definition
AI	Machines doing tasks that need human intelligence
Machine Learning	Algorithms that learn patterns from data
Deep Learning	ML built with many-layered neural networks
NLP	Helping machines work with human language
Tokenization	Splitting text into tokens a model can process
Embedding	Mapping words or tokens to number vectors
Word2Vec	Local-context, static word embeddings
GloVe	Global co-occurrence, static word embeddings
Transformer	The 2017 attention-based architecture behind modern AI
Self Attention	Each token attends to the other tokens
Multi-Head Attention	Several parallel attention heads for different relations
Encoder	Transformer part for understanding, for example BERT
Decoder	Transformer part for generation, for example GPT
LLM	Large Transformer trained on massive text data
AI Agent	LLM plus tools plus memory plus planning, acting on its own

References (Official Sources)

- Attention Is All You Need. Vaswani et al., 2017. arXiv:1706.03762.
- OpenAI API reference, temperature parameter. Range 0 to 2, default 1.
- Anthropic API reference, temperature parameter. Range 0 to 1, default 1.